

Optimal Control Recovery from Quadcopter Inversion with Perturbations

Luke G. Kiziah^{1†}

[†]Department of Mechanical and Aerospace Engineering, North Carolina State University, Raleigh, North Carolina, United States.

¹lgkiziah@ncsu.edu

May 1st, 2026

Abstract

This paper provides a fundamentals based analysis to optimal guidance, particularly for strict input constraints and large perturbations. The nonlinear dynamics of a quadrotor UAV are derived and modeled. A comprehensive suite of control design techniques—including the discrete and continuous Linear Quadratic Regulator (LQR), finite-horizon LQR, Time-Varying LQR (TVLQR), and Model Predictive Controllers (MPC)—are derived and applied. The comparative analysis provides a rigorous baseline, on the viability of computationally efficient optimal controllers and their application in a challenging scenario—the inverted quadcopter.

A Introduction

Control theory has undergone significant advancement from early control through the pre-classical period, classical period, and continuing now with modern control [1]. Modern control leverages state-space representations to systematically address observability, controllability, robustness, and optimality in multi-input multi-output (MIMO) systems [1, 2]. The state-space approach relies on precise mathematical models; accordingly, dynamic modeling has evolved from fundamental mechanics and traditional system identification to modern, AI-driven methods capable of adapting to complex plant dynamics [3, 4]. One successful artifact of modern control is MPC, which is widely employed in constrained optimal control problems from rocket guidance [5] to chemical process control [6]. To thoroughly explain trajectory optimization, this paper will steadily derive optimal control methodologies of increasing complexity and apply them to a real world system.

Plant Models

In the controller design process linear models often prove valuable, enabling straightforward optimal control design given a state space representation of the form in Equation 1; however, real systems

are often highly non-linear in the form of Equation 2.

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} \quad (1)$$

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}) \quad (2)$$

There may also be disturbance, noise, and time varying dynamical effects, though these will be addressed later. In any case, most control formulations are designed for linear models. Therefore, a standard approach when dynamics are known is to linearize the nonlinear equations of motion to the 1st order partial derivatives in a Taylor series expansion, also called Jacobian linearization, around the desired operating point as shown in Equations 3 and 4.

$$\dot{\mathbf{x}} = \dot{\bar{\mathbf{x}}} + \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{(\bar{\mathbf{x}}, \bar{\mathbf{u}})} (\mathbf{x} - \bar{\mathbf{x}}) + \left. \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \right|_{(\bar{\mathbf{x}}, \bar{\mathbf{u}})} (\mathbf{u} - \bar{\mathbf{u}}) + H.O.T \quad (3)$$

$$\dot{\tilde{\mathbf{x}}} \approx \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{(\bar{\mathbf{x}}, \bar{\mathbf{u}})} \tilde{\mathbf{x}} + \left. \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \right|_{(\bar{\mathbf{x}}, \bar{\mathbf{u}})} \tilde{\mathbf{u}} = \mathbf{A}\tilde{\mathbf{x}} + \mathbf{B}\tilde{\mathbf{u}} \quad (4)$$

The addition of a bar represents an operating point and tilde represents deviation from that point. Note, numerator layout is used throughout this paper for vector and matrix derivatives to allow direct mapping from Jacobian to state space matrices. If the operating point is also an equilibrium point, i.e. if $\mathbf{f}(\bar{\mathbf{x}}, \bar{\mathbf{u}}) = 0$, then $\bar{\mathbf{u}}$ can be solved by inversion. Alternatively, as in the case of TVLQR which will be addressed later, a desired trajectory and control sequence may already be known; in which case the nonlinear equations are linearized around the desired trajectory.

Additionally, with the ever-increasing significance of digital control, obtaining a discrete model is often useful. It may be desirable to find, or reasonably approximate, a non-linear discrete system in the form of Equation 5. A standard direct numerical integrator is the Fourth-Order Runge-Kutta (RK4) which is employed and compared in [7], numerical integration formulas are given in Appendix A.

$$x_{k+1} = f_d(x_k, u_k) \quad (5)$$

Alternatively, it may be preferred to linearize the nonlinear continuous dynamics and then discretize. By taking the Laplace transform of Equation 1, solving for $X(s)$ and taking the inverse transform, a general exact solution to a continuous linear plant over a discrete time interval T_s from $t_0 = kT_s$ to $t = (k+1)T_s$ is given by Equation 6.

$$\mathbf{x}((k+1)T_s) = e^{\mathbf{A}T_s} \mathbf{x}(kT_s) + \int_{kT_s}^{(k+1)T_s} e^{\mathbf{A}((k+1)T_s - \tau)} \mathbf{B}\mathbf{u}(\tau) d\tau \quad (6)$$

If the dynamics and input are constant during the period from $kT_s \leq t < (k+1)T_s$ the discrete time state space model is given by Equation 7.

$$\begin{aligned} \mathbf{x}_{k+1} &= e^{\mathbf{A}T_s} \mathbf{x}_k + \int_0^{T_s} e^{\mathbf{A}\tau} d\tau \cdot \mathbf{B}\mathbf{u}_k \\ &= \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k \end{aligned} \quad (7)$$

For computing these discrete time matrices, the general best practice is to form an augmented state space representation as in Equation 8 which results in Equation 9 from which $\mathbf{A}_d$ and $\mathbf{B}_d$ can be

extracted.

$$\begin{aligned} \begin{bmatrix} \dot{\mathbf{x}} \\ \dot{\mathbf{u}} \end{bmatrix} &= \begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{u} \end{bmatrix} \\ &= \mathbf{M} \begin{bmatrix} \mathbf{x} \\ \mathbf{u} \end{bmatrix} \end{aligned} \quad (8)$$

$$\begin{aligned} \begin{bmatrix} \mathbf{x}_{k+1} \\ \mathbf{u}_{k+1} \end{bmatrix} &= e^{\mathbf{M}T_s} \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{A}_d & \mathbf{B}_d \\ \mathbf{0} & \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix} \end{aligned} \quad (9)$$

This can be quickly carried out using *c2d(ContinuousSystem, T_s)*¹ which defaults to zero order hold and the same solution, or by using *c2dFREE(A, B, T_s)* in *OCP496* [8].

Continuous Infinite Horizon LQR

The linear quadratic regulator or LQR is a commonly employed optimal control policy for a linear plant in the form of Equation 1. LQR is optimal in that it minimizes a cost function. Continuous infinite horizon LQR will be addressed first in which the cost function takes the form of Equation 10.

$$\begin{aligned} J &= \int_0^\infty (\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u}) dt \\ &= \int_0^\infty L(\mathbf{x}, \mathbf{u}) dt \end{aligned} \quad (10)$$

$L(\mathbf{x}, \mathbf{u})$ represents the running cost where the chosen constant weighting matrices for state error and control effort $\mathbf{Q} \succeq 0$ and $\mathbf{R} \succ 0$. Additionally, $(A, Q^{1/2})$ must be detectible and (A, B) must be stabilizable for the existence of an optimal stabilizing solution. In an effort to derive the Hamilton-Jacobi-Bellman equation in pursuit of LQR, a value function is defined which represents the cost-to-go from a particular state if an optimal control sequence is used, shown in Equation 11.

$$\begin{aligned} V(\mathbf{x}) &= \min_u \int_0^\infty L(\mathbf{x}, \mathbf{u}) dt \\ &= \mathbf{x}^T \mathbf{P} \mathbf{x} \\ &= \min_u (L(\mathbf{x}, \mathbf{u}) dt) + V(\mathbf{x} + \dot{\mathbf{x}} dt) \end{aligned} \quad (11)$$

¹Using *c2d* requires MATLAB's Control System Toolbox

Where $\mathbf{P}$ denotes the optimal cost matrix and $V(\mathbf{x} + \dot{\mathbf{x}}dt) \approx V(\mathbf{x}) + \nabla V \dot{\mathbf{x}}dt$. The Hamilton-Jacobi-Bellman equation is then given by Equation 12.

$$\begin{aligned} 0 &= \min_u (L(\mathbf{x}, \mathbf{u}) + \dot{\mathbf{x}}^T \mathbf{P} \mathbf{x} + \mathbf{x}^T \mathbf{P} \dot{\mathbf{x}}) \\ &= \min_u (\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u} + (\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u})^T \mathbf{P} \mathbf{x} + \mathbf{x}^T \mathbf{P} (\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u})) \\ &= \min_u (\mathbf{x}^T (\mathbf{Q} + 2\mathbf{P} \mathbf{A}) \mathbf{x} + 2\mathbf{x}^T \mathbf{P} \mathbf{B} \mathbf{u} + \mathbf{u}^T \mathbf{R} \mathbf{u}) \end{aligned} \quad (12)$$

Because Equation 12 represents the minimum in the cost landscape, its derivative with respect to u minimizes the cost resulting in Equation 13 which defines the optimal control.

$$\begin{aligned} \mathbf{u}^* &= -\mathbf{R}^{-1} \mathbf{B}^T \mathbf{P} \mathbf{x} \\ &= -\mathbf{K}^* \mathbf{x} \end{aligned} \quad (13)$$

Here, $*$ is used to represent optimality. Substituting Equation 13 back into the Hamilton-Jacobi-Bellman equation results in Equation 14, the Continuous-time Algebraic Ricatti equation.

$$\mathbf{Q} + \mathbf{A}^T \mathbf{P} + \mathbf{P} \mathbf{A} - \mathbf{P} \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T \mathbf{P} = 0 \quad (14)$$

Equation 14 is solved for $\mathbf{P}$ which can be applied to Equation 13 to obtain $\mathbf{u}^*$. Furthermore, it can be shown that $V > 0$ and $\dot{V} \leq 0$, making the solution Lyapunov stable if the linear dynamics are maintained without disturbance.

In MATLAB the resulting LQR gains can be computed with $lqr(A, B, Q, R)^2$.

Continuous Finite Horizon LQR

Continuous time finite horizon LQR is treated similarly to the infinite horizon case with the addition of a horizon such that cost is described by Equation 15, resulting in a time varying optimal weighting matrix $P(t)$ and correspondingly $K(t)^*$ that can be solved for with the differential Riccati equation, Equation 16. Equation 16 is backwards integrated from the terminal condition of Equation 17 to present.

$$J = \int_{t_0}^{t_f} (x^T Q x + u^T R u) dt + x(t_f)^T Q_f x(t_f) \quad (15)$$

$$-\dot{P}(t) = Q + A^T P + P A - P B R^{-1} B^T P \quad (16)$$

$$P(t_f) = Q_f \quad (17)$$

Note that $Q_f \succeq 0$, $Q \succeq 0$, and $R \succ 0$ so that the cost is convex in u , guaranteeing a unique optimal control law. Once $P(t)$ is known, the optimal input over the entire horizon is described by Equation 18.

$$\begin{aligned} u(t)^* &= -R^{-1} B^T P(t) x \\ &= -K^*(t) x \end{aligned} \quad (18)$$

²Using $lqr()$ requires the Control System Toolbox

It can additionally be shown that if (A, B) is stabilizable and if $(A, Q^{1/2})$ is detectible then as $t_f \rightarrow \infty$, $\dot{P} \rightarrow 0$, meaning the finite horizon LQR solution approaches the infinite horizon LQR solution for large t_f as expected.

For computing the solution by numerical integration *ContinuousFiniteHorizonLQR* in *OCP496* is used [8]. In practice, unless integration is carried out explicitly, discrete finite horizon LQR is generally recommended [9].

Continuous TVLQR

TVLQR can be using to design a controller that stabilizes a nonlinear system around a previously computed, feasible, desired trajectory $x_d(t), u_d(t)$ where $\dot{x}_d = f(x_d(t), u_d(t))$. Equation 4 can then be applied where again tilde represents deviation from bar of the same variable, but now $\tilde{x}$ and $\tilde{u}$ represent the desired trajectory³. Therefore, the state space model is linearized around the desired trajectory, see Equations 19 and 20.

$$A(t) = \frac{\partial f}{\partial x}|_{(x_d(t), u_d(t))} \quad (19)$$

$$B(t) = \frac{\partial f}{\partial u}|_{(x_d(t), u_d(t))} \quad (20)$$

Fortunately, in TVLQR if a desired input $u_d(t)$ is already known then the problem can be defined similar to finite horizon LQR but with x and u replaced with $\tilde{x}$ and $\tilde{u}$ respectively. Correspondingly the cost, time-varying Riccati equation, and optimal control law are given by Equations 21, 22, and 23 making the final controller follow Equation 24.

$$J = \int_{t_0}^{t_f} (\tilde{x}^T Q \tilde{x} + \tilde{u}^T R \tilde{u}) + \tilde{x}^T(t_f) Q_f \tilde{x}(t_f) \quad (21)$$

$$-\dot{P}(t) = Q + A^T(t)P + PA(t) - PB(t)R^{-1}B^T(t)P \quad (22)$$

$$\begin{aligned} \tilde{u}^*(t) &= -R^{-1}B^T(t)P(t)\tilde{x} \\ &= -K^*(t)\tilde{x} \end{aligned} \quad (23)$$

$$u^*(t) = u_d(t) - K^*(t)\tilde{x} \quad (24)$$

For an unchanging, once computed, desired trajectory, TVLQR is said to be open-loop in trajectory and closed-loop in deviations. However, because linearization is carried out around a nominal trajectory, if $\tilde{x}$ becomes too large the linearization around the desired trajectory becomes invalid in approximating dynamics, resulting in robustness issues. Therefore, it is desirable to update the nominal trajectory and resolve TVLQR repeatedly for robustness, feasibility, and optimality. Alternatively, a simpler but similarly more robust approach—when compared to standard TVLQR—is to repeatedly solve LQR or finite horizon LQR with linearized state space matrices evaluated at the current state but maintaining $\bar{x} = x_d$ and $\bar{u} = u_d$.

³Note, Equations 3 and 4 can be applied to all LQR formulations, but the distinction here is that linearization is carried out around a desired trajectory instead of one state input pair. In all cases their respective Riccati equations may be applied where x is replaced with $\tilde{x}$; though, u can be treated differently depending on the intended goal. For traditional LQR where it is desirable to minimize control effort, replacing u with $\tilde{u}$ produces the undesired effect of contributing 0 cost even if u is nonzero, so special care should be taken in both the problem setup and implementation.

For carrying out the computation *ContinuousTimeVaryingLQR* in *OCP496* is used [8]. Similar to in continuous finite horizon LQR, the discrete version should in general be used.

Discrete Infinite Horizon LQR

The discrete infinite horizon LQR cost function is defined in Equation 25 for a system represented by Equation 7.

$$\begin{aligned} J &= \sum_{k=0}^{\infty} (x_k^T Q x_k + u_k^T R u_k) \\ &= \sum_{k=0}^{\infty} L(x_k, u_k) \end{aligned} \quad (25)$$

Similar to the continuous time case it is required that $Q \succeq 0$, $R \succ 0$, (A_d, B_d) is stabilizable, and that $(A_d, Q^{1/2})$ is detectible. The value function is defined by Equation 26, where the final form is called the Bellman equation.

$$\begin{aligned} V(x_k) &= \min_u \sum_{k=0}^{\infty} L(x_k, u_k) \\ &= x_k^T P x_k \\ &= \min_u (L(x_k, u_k) + V(x_{k+1})) \end{aligned} \quad (26)$$

Equation 26 can further be expanded and simplified to the Discrete-time Algebraic Riccati Equation given by Equation 27 where the function was minimized similar to the continuous time case by differentiating with respect to u_k to find Equation 28.

$$P = Q + A_d^T P A_d - A_d^T P B_d (R + B_d^T P B_d)^{-1} B_d^T P A_d \quad (27)$$

$$\begin{aligned} u^* &= -(R + B_d^T P B_d)^{-1} B_d^T P A_d x \\ &= -K_d^* x \end{aligned} \quad (28)$$

Lyapunov stability can be shown similarly to the continuous time case.

The discrete LQR gains can be computed using *dlqr*(A_d, B_d, Q, R) or directly from continuous state space matrices using *lqrd*(A, B, Q, R, T_s)⁴.

Discrete Finite Horizon LQR

Discrete finite horizon LQR optimizes control inputs for a system represented by Equation 7 over a horizon, N , where the cost function is given by Equation 29.

$$J = \sum_{k=0}^{N-1} (x_k^T Q x_k + u_k^T R u_k) + x_N^T Q x_N \quad (29)$$

Just like the continuous case, we require $Q_f \succeq 0$, $Q \succeq 0$, and $R \succ 0$ so that the cost is convex in u , guaranteeing a unique optimal control law. The Discrete Finite Horizon Riccati Recursion then

⁴Both *dlqr* and *lqrd* require MATLAB's Control System Toolbox.

follows Equation 30, which can be stepped backwards from the terminal state of Equation 31 to present.

$$P_k = Q + A_d^T P_{k+1} A_d - A_d^T P_{k+1} B_d (R + B_d^T P_{k+1} B_d)^{-1} B_d^T P_{k+1} A_d \quad (30)$$

$$P_N = Q_f \quad (31)$$

The time varying weighting matrix P_k results in a time varying control law given by Equation 32.

$$\begin{aligned} u_k^* &= -(R + B_d^T P_{k+1} B_d)^{-1} B_d^T P_{k+1} A_d x_k \\ &= -K_k^* x_k \end{aligned} \quad (32)$$

Similar to the continuous time case, if (A_d, B_d) is stabilizable and if $(A_d, Q^{1/2})$ is detectable then as $N \rightarrow \infty$ the finite horizon solution approaches the infinite horizon one.

For carrying out the computation *DiscreteFiniteHorizonLQR* in *OCP496* is used [8].

Discrete TVLQR

Similar to the continuous case, the goal is to linearize and stabilize the system around the desired trajectory x_k^d, u_k^d . Assuming the dynamics are in the form of Equation 2, these continuous dynamics will be linearized about the nominal trajectory at each time step $k \in (0, 1, 2, \dots, N-1)$ giving Equations 33 and 34.

$$A_k = \frac{\partial f_k}{\partial x} \big|_{(x_k^d, u_k^d)} \quad (33)$$

$$B_k = \frac{\partial f_k}{\partial u} \big|_{(x_k^d, u_k^d)} \quad (34)$$

These continuous time linear state space matrices can then be discretized using a method like Equation 8 explained earlier. Letting $\tilde{x}_k$ replace x_k and $\tilde{u}_k$ replace u_k in the discrete finite horizon equations; the cost, time-varying Riccati recursion, and optimal control law are given by Equations 35, 36, and 37 given Equation 31, making the final controller follow Equation 38.

$$J = \sum_{k=0}^{N-1} (\tilde{x}_k^T Q \tilde{x}_k + \tilde{u}_k^T R \tilde{u}_k) + \tilde{x}_N^T Q_f \tilde{x}_N \quad (35)$$

$$P_k = Q + A_{k_d}^T P_{k+1} A_{k_d} - A_{k_d}^T P_{k+1} B_{k_d} (R + B_{k_d}^T P_{k+1} B_{k_d})^{-1} B_{k_d}^T P_{k+1} A_{k_d} \quad (36)$$

$$\begin{aligned} \tilde{u}_k^* &= -(R + B_{k_d}^T P_{k+1} B_{k_d})^{-1} B_{k_d}^T P_{k+1} A_{k_d} \tilde{x}_k \\ &= -K_k^* \tilde{x}_k \end{aligned} \quad (37)$$

$$u_k = u_k^d - K_k^* \tilde{x}_k \quad (38)$$

Similar to the continuous time case, it is desirable to add robustness by modifying or repeatedly updating the control law.

For carrying out the computation *DiscreteTimeVaryingLQR* in *OCP496* is used [8].

Linear Model Predictive Control (Single Shooting)

Model Predictive Control (MPC) is a widely adopted control method that optimizes trajectories by forecasting based on a system model, and which handles state and input constraints. This

discussion begins with the most simple form of MPC, that being for a linear unconstrained system with dynamics of the form in Equation 7. The trajectory of states over a horizon N follows Equation 39, or equivalently in matrix form, Equation 40 (A and B are used to denote the discrete time state space matrices).

$$\begin{aligned} x_{k+1} &= Ax_k + Bu_k \\ x_{k+2} &= Ax_{k+1} + Bu_{k+1} \\ &= A^2x_k + ABu_k + Bu_{k+1} \end{aligned} \quad (39)$$

$$\begin{aligned} &\dots \\ x_{k+N} &= A^Nx_k + A^{N-1}Bu_k + A^{N-2}Bu_{k+1} + \dots + Bu_{k+N-1} \end{aligned}$$

$$\begin{bmatrix} x_{k+1} \\ x_{k+2} \\ \dots \\ x_{k+N} \end{bmatrix} = \begin{bmatrix} A \\ A^2 \\ \dots \\ A^N \end{bmatrix} x_k + \begin{bmatrix} B & 0 & \dots & \dots & 0 \\ AB & B & 0 & \dots & 0 \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ A^{N-1}B & A^{N-2}B & \dots & \cdot & B \end{bmatrix} \begin{bmatrix} u_k \\ u_{k+1} \\ \dots \\ u_{k+N-1} \end{bmatrix} \quad (40)$$

$$X = \mathcal{A}x_k + \mathcal{B}U$$

Dimensionality follows $X \in \mathbb{R}^{N \cdot n}$ and $U \in \mathbb{R}^{N \cdot m}$, maintaining $x_k \in \mathbb{R}^n$ and $u_k \in \mathbb{R}^m$. The objective function is defined according to Equation 41 with the state and control weighting matrices given by $Q_h = \text{diag}(Q_1, Q_2, \dots, Q_f)$ and $R_h = \text{diag}(R, \dots, R)$ respectively; Q_h and R_h are computed with *bigWeighting* in *OCP496* [8].

$$\begin{aligned} J &= \min_u X^T Q_h X + U^T R_h U \\ &= \min_u U^T (\mathcal{B}^T Q_h \mathcal{B} + R_h) U + 2x_k^T (\mathcal{A}^T Q_h \mathcal{B}) U + x_k^T \mathcal{A}^T Q_h \mathcal{A} x_k \\ &= \min_u \frac{1}{2} U^T H U + x_k^T F U + x_k^T \mathcal{A}^T Q_h \mathcal{A} x_k \end{aligned} \quad (41)$$

$H = 2\mathcal{B}^T Q_h \mathcal{B} + 2R_h$ is the quadratic objective term, known as the Hessian, and $F = 2\mathcal{A}^T Q_h \mathcal{B}$ is an intermediary variable defined, rather than the standard linear objective term, for simplification in relating the MPC solution to the gain matrix obtained in LQR. The latter form of Equation 41 is useful in computing the minimization and aligns closely to standard quadratic program (QP) form which will be used later. In the absence of constraints, if an optimal stabilizing solution exists, the gradient of the objective function with respect to U evaluated at the optimal input sequence follows Equation 42.

$$\begin{aligned} 0 = \nabla_U J(U^*) &= \frac{1}{2} U^{*T} (H + H^T) + x_k^T F \\ &= H U^* + F^T x_k \end{aligned} \quad (42)$$

The gradient is zero because U^* lies at the minimum of a convex function when no constraints are present. The latter form of Equation 42 holds if Q_h and R_h are symmetric. The optimal control

sequence then follows Equation 43.

$$\begin{aligned} U^* &= -H^{-1}F^T x_k \\ &= -\mathcal{K}_{MPC} x_k \\ &= -H^{-1}f \end{aligned} \tag{43}$$

The middle form of Equation 43 defines the optimal control sequence by a matrix of gains over the trajectory multiplied by the initial state. Unsurprisingly, the control sequence matches exactly with that of discrete time finite horizon LQR in the absence of disturbances and constraints assuming horizon, weighting matrices, etc. are kept equal—both optimize over the same trajectory with equivalent objectives. Therefore, extracting the first m rows of U^* and $\mathcal{K}_{MPC}$, gives the first input with a gain matrix equivalent to that in the finite horizon LQR case, Equation 44.

$$\begin{aligned} u_k^* &= -K_{MPC_k} x_k \\ &= -K_{LQR_k} x_k \end{aligned} \tag{44}$$

Additionally, the final form of Equation 43 introduces the linear objective term, f useful in defining a QP.

In MPC, the problem is typically resolved at each time step such that only the first input in the sequence is applied each time. The unconstrained optimization problem can be solved in MATLAB with $U^* = H \setminus f$. Though for efficiency, if H is symmetric positive definite, solving should generally be carried out using the Cholesky factorization, which is given by *Cholesky* in *OCP496* [8]. Alternatively, the problem can be posed and solved from standard QP form given by Equation 45; the solution is equivalent because minimization is only with respect to U , and the function being minimized is simply the U dependent portion of the original objective function, $J(U)$.

$$\min_U \frac{1}{2} U^T H U + f^T U \tag{45}$$

Equation 45 can be solved with *quadprog*⁵. The QP carries over nicely to constraints, which will be covered next.

Real world problems often have state and input constraints; therefore, linear MPC is subsequently extended to linear equality and inequality constraints. In the presence of active constraints Equation 42 is no longer valid—the unconstrained optimal solution becomes infeasible. Therefore, the constrained optimal solution exists where the objective function from Equation 41 cannot be decreased in any feasible direction from U^* , where U^* now denotes the optimal constrained solution. To enforce feasibility a constraint equality is defined in Equation 46.

$$g(U) = 0 \tag{46}$$

Thereby, any admissible δU from U^* must lie in the tangent space of the constraint surface, meaning $\nabla_U g(U^*) \cdot \delta U = 0$ since g is a level set. For optimality, $\nabla_U J(U^*) \cdot \delta U = 0$ because $\nabla_U J(U)$ can not exist along any feasible direction at optimum. Considering the constraint equation and objective function at U^* , their gradients are therefore parallel vectors giving Equation 47, which provides the

⁵Use of *quadprog* requires the Optimization Toolbox.

optimality condition.

$$\nabla_U J(U^*) = -\lambda \nabla_U g(U^*) \quad (47)$$

In fact, this result emerges from Lagrange multipliers where the Lagrangian is correspondingly given by Equation 48.

$$L(U, \lambda) = J(U) + \lambda g(U) \quad (48)$$

If equality constraints are defined according to Equation 49 and inequality constraints according to Equation 50, based on the optimality condition of Equation 47 and letting $J(U)$ follow the QP form in Equation 45, the matrix representing all simultaneous equations at optimum is given by Equation 51—known as the Karush-Kuhn-Tucker (KKT) system.

$$A_{eq}U = b_{eq} \quad (49)$$

$$GU \leq h \quad (50)$$

$$\begin{bmatrix} H & A_{eq}^T & G_A^T \\ A_{eq} & 0 & 0 \\ G_A & 0 & 0 \end{bmatrix} \begin{bmatrix} U \\ \nu \\ \lambda \end{bmatrix} = \begin{bmatrix} -f \\ b_{eq} \\ h_A \end{bmatrix} \quad (51)$$

The matrix G_A and vector h_A are the active inequality constraints from G and h . Different from equality constraints, where Equation 49 always requires equality, if a given inequality $g_i(U^*) < 0$ that inequality is non-active; therefore it is not part of G_A and h_A . In general, the active constraints are initially unknown. Therefore, an algorithm is followed to iteratively modify the active set, solve the KKT system, check if all constraints are satisfied, and that the active set is fully active. The equality Lagrange multipliers, ν , do not need to be checked because they span $\nu \in \mathbb{R}$, but the inequality multipliers $\lambda \geq 0$ when active; hence, $\lambda_i < 0$ are removed from the active set. MATLAB's function *quadprog* does this automatically. For control engineers, the relevant part in solving a QP is therefore initializing the solver. Accordingly, three types of constraints are mentioned here: State bounds, input bounds, and enforcing a terminal state (explicit dynamic constraints are omitted for brevity but are later addressed in multiple shooting MPC). Creating inequality constraints for input bounds in single shooting is trivial because they are naturally a function of U and are given by Equation 52 when linear.

$$G_u U \leq h_u \quad (52)$$

The number of control bounds, c , results in $G_u \in \mathbb{R}^{c \times (N \cdot m)}$ and $h_u \in \mathbb{R}^c$. State bounds are defined according to Equation 53 when linear, with $\mathcal{G}_x \in \mathbb{R}^{q \times N \cdot n}$ and $h_x \in \mathbb{R}^q$ where there are q state bounds.

$$\mathcal{G}_x X \leq h_x \quad (53)$$

To apply Equation 45, Equation 53 is converted into a function of U . Substituting Equation 40 into Equation 53 results in the desired form, Equation 54.

$$\begin{aligned} \mathcal{G}_x B U &\leq h_x - \mathcal{G}_x A x_k \\ G_x U &\leq h_x \end{aligned} \quad (54)$$

The state and input inequality constraints are concatenated to form Equation 50, where $G_a \subseteq G$ and $h_a \subseteq h$. Lastly, enforcing a terminal state constraint results in an equality following Equation

49. From Equation 40, the last n rows are extracted from $\mathcal{A}$ and $\mathcal{B}$, $\mathcal{A}_{N^{th}}$ and $\mathcal{B}_{N^{th}}$ giving Equation 55.

$$\begin{aligned} x_{k+N} &= A^N x_k + A^{N-1} B u_k + A^{N-2} B u_{k+1} + \dots + B u_{k+N-1} \\ &= \mathcal{A}_{N^{th}} x_k + \mathcal{B}_{N^{th}} U \\ &= x_{target} \end{aligned} \tag{55}$$

Consequently, the equality constraint matrix and vector are defined by Equations 56 and 57.

$$A_{eq} = \mathcal{B}_{N^{th}} \tag{56}$$

$$b_{eq} = x_{target} - \mathcal{A}_{N^{th}} x_k \tag{57}$$

The MPC form thus far developed is known as the single shooting method. With the objective function defined in Equation 45, the dynamics were enforced implicitly within the objective function. Therefore, depending on the objective, dynamics may need to be enforced differently—for example if $J = U^T R_h U$ dynamics must be enforced as equality constraints. Additionally, the terminal state equality constraint formed from Equations 55, 56, and 57 results in poor numerical conditioning, because x_{k+N} is dependent on the chain of all previous actions and dynamics starting from x_k —this worsens as the horizon grows. The same applies to how dynamics and state inequality constraints are enforced in single shooting.

Multiple Shooting MPC and NMPC

Multiple shooting simultaneously optimizes over X and U by reformulating the QP in z . z is given by Equation 58.

$$z = \begin{bmatrix} x_k \\ x_{k+1} \\ \dots \\ x_{k+N} \\ u_k \\ u_{k+1} \\ \dots \\ u_{k+N-1} \end{bmatrix} \tag{58}$$

Here, MPC will be formulated rather generally for solving via multiple shooting, and each use case will be mentioned. The problem of tracking a known reference trajectory is also considered. Additionally, the system may be nonlinear of the form in Equation 5 and contain nonlinear constraints 59 and 60.

$$a(z) = 0 \tag{59}$$

$$g(z) \leq 0 \tag{60}$$

In multiple shooting, dynamics are defined as equality constraints locally, resulting in larger, but sparse and well conditioned constraint and Hessian matrices as compared to single shooting. The standard Time Varying LQR objective function is used, but reformulated for MPC as shown in

Equation 61.

$$\begin{aligned} J &= \min_{U, X} (X - X^{ref})^T Q_h (X - X^{ref}) + (U - U^{ref})^T R_h (U - U^{ref}) \\ &= \min_z \frac{1}{2} z^T H \delta z + f^T z \end{aligned} \quad (61)$$

The Hessian⁶ is symmetric positive semidefinite and given by Equation 62; the linear objective⁷ is given by Equation 63.

$$H = 2 \begin{bmatrix} 0 & \mathbf{0} \\ Q_h & R_h \\ \mathbf{0} & R_h \end{bmatrix} \quad (62)$$

$$f = -2 \begin{bmatrix} 0 \\ Q_h X^{ref} \\ R_h U^{ref} \end{bmatrix} \quad (63)$$

The minimization is subject to dynamics in Equation 64.

$$\delta x_{k+1} = A_k \delta x_k + B_k \delta u_k + r_k \quad (64)$$

The error dynamics in Equation 64 include a 1st order linearized dynamics correction term $r_k = f_k(x_k^{(i)}, u_k^{(i)}) - x_{k+1}^{(i)}$.

For an LTI system of course no dynamics linearization occurs, but r_k still serves to correct the reference trajectory if it does not follow the dynamics exactly, (i) becomes the reference trajectory. For LTI systems $A_k = A$ and $B_k = B$ are constant linear state space matrices, $\delta x_k = x_k - x_k^{ref}$, and $\delta u_k = u_k - u_k^{ref}$. If constraints are nonlinear they can be linearized following Equations 65 and 66.

$$a(z) \approx a(z^{traj}) + \frac{\partial a}{\partial z} \Big|_{z^{traj}} (z - z^{traj}) \quad (65)$$

$$g(z) \approx g(z^{traj}) + \frac{\partial g}{\partial z} \Big|_{z^{traj}} (z - z^{traj}) \quad (66)$$

Applying the conditions in Equations 59 and 60 to the Jacobian linearizations of Equations 65 and 66 results in the familiar linear constraint form of Equations 67 and 68.

$$A_{eq} z = b_{eq} \quad (67)$$

$$Gz \leq h \quad (68)$$

In NMPC an initial optimal trajectory guess is specified, dynamics and constraints are linearized around the guessed optimal trajectory, the resulting QP is solved, and the previous guess is successively updated towards the current optimal QP solution—taking Newton type steps towards convergence on the local optimal trajectory. This is also known as Sequential Quadratic Programming (SQP), and convergence ensures constraint satisfaction and optimality. In NMPC (i) represents the i^{th} guessed optimal trajectory. Accordingly, $\delta x_k = x_k - x_k^i$, $\delta u_k = u_k - u_k^i$, the constraint linearizations come from Equations 65 and 66 where $traj$ is replaced with i , and the dynamics are

⁶The 0 in the first entry of H is associated with x_0 and is therefore $0 \in \mathbb{R}^{n \times n}$.

⁷The 0 in the first entry of f is associated with x_0 and is therefore $0 \in \mathbb{R}^n$.

linearized from Equation 5 to Equation 7 around the guessed trajectory. Dynamics are enforced via equality constraints following Equation 69, in which sparsity and locality is apparent.

$$\begin{bmatrix}
 I & 0 & 0 & 0 & \dots & \dots & \dots & \dots & \dots & 0 \\
 -A_k & I & 0 & 0 & \dots & \dots & -B_k & 0 & \dots & 0 \\
 0 & -A_{k+1} & I & 0 & \dots & \dots & \dots & -B_{k+1} & 0 & \dots \\
 \vdots & \vdots & \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\
 0 & \dots & \dots & -A_{k+N-1} & I & 0 & 0 & \dots & -B_{k+N-1} & \dots
 \end{bmatrix}
 \begin{bmatrix}
 x_k \\
 x_{k+1} \\
 x_{k+2} \\
 \dots \\
 x_{k+N} \\
 u_k \\
 \dots \\
 u_{k+N-1}
 \end{bmatrix}
 =
 \begin{bmatrix}
 x_{initial} \\
 c_k \\
 c_{k+1} \\
 \dots \\
 c_{k+N-1}
 \end{bmatrix}$$

$$A_{eqfd} z = b_{eqfd} \quad (69)$$

The constants $c_k = f_d(x_k^{(i)}, u_k^{(i)}) - A_k x_k^{(i)} - B_k u_k^{(i)}$ result from absolute dynamics form, as opposed to error dynamics in Equation 64, making a 1st order linearized dynamics correction equivalent to r_k in absolute dynamics form. If a terminal state is also enforced, its corresponding equality constraint is given by Equation 70.

$$\begin{bmatrix}
 0 & \dots & I & 0 & \dots & 0
 \end{bmatrix}
 \begin{bmatrix}
 x_k \\
 \dots \\
 x_{k+N} \\
 u_k \\
 \dots \\
 u_{k+N-1}
 \end{bmatrix}
 = x_f$$

$$A_{eqf} z = b_{eqf} \quad (70)$$

The equality constraints are then concatenated into Equation 67. For inequalities, the extension of Equations 52 and 53 to Equation 68 is relatively straightforward, and is therefore omitted for brevity. This formulation creates local QPs that can be computed by iteratively forming the KKT system, solving, checking constraints, and updating active constraints just like in constrained linear single shooting MPC.

Real-Time Iteration

Due to the computational complexity of running an SQP until convergence at the sampling or controller rate in real time, a computationally affordable alternative known as Real-Time Iteration (RTI) is used to approximate the converged solution. According to [7] if RTI is "warm-started at the global optimum", "sampling frequency is sufficiently high", "there are no jumps in the reference and the state", "the OCP underlying the NMPC problem fulfills Second Order Sufficient Conditions (SOSC) for every feasible initial condition", and "the global optimum depends continuously on the initial state and reference" the RTI solution follows the global solution of the non-convex problem. While this work does not attempt to prove these statements or the satisfaction of any one of them in the implementation proposed herein, this work uses these assumptions as guidelines in development of optimal but practical guidance. Following from the previous subsection, RTI is implemented

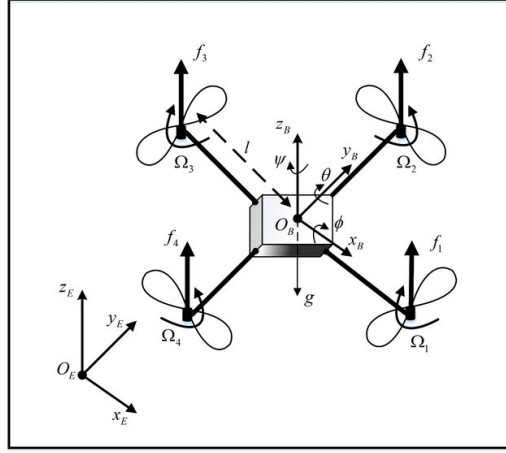


Figure 1: Plus Configuration Quadcopter Diagram [10]

as a single full Newton step of the SQP at each time step. RTI is initialized with a precomputed reference trajectory which is also (partly) used as the initial guess, from which receding horizon MPC shifts the reference trajectory and RTI shifts the most recent guess forward while incorporates new state estimates according to Algorithm 3 in [7]. The preparation and feedback phase algorithmic choices from [7] are used to practically implement NMPC. We refer to this method as Receding RTI. Shrinking RTI is also used in the analysis which follows a similar process, but where the horizon is fixed in time.

Quadrotor UAV

The quadcopter is defined with a right-handed coordinate system and according to a plus configuration, such that body axes pass through the rotor rotational axes as seen in Figure 1. Translational acceleration shall be described in the inertial frame and angular acceleration in the body frame. A successive ZYX rotation sequence (Euler Angles) is used to convert a body vector to an inertial vector following Equations 71 and 72, where $\sin$ and $\cos$ are given by s and c .

$${}^I \mathbf{v} = {}^I_B \mathbf{R}(\phi, \theta, \psi) {}^B \mathbf{v} \quad (71)$$

$$\begin{aligned} {}^I_B \mathbf{R}(\phi, \theta, \psi) &= \mathbf{R}_z(\psi) \mathbf{R}_y(\theta) \mathbf{R}_x(\phi) \\ &= \begin{bmatrix} c(\psi) & -s(\psi) & 0 \\ s(\psi) & c(\psi) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c(\theta) & 0 & s(\theta) \\ 0 & 1 & 0 \\ -s(\theta) & 0 & c(\theta) \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & c(\phi) & -s(\phi) \\ 0 & s(\phi) & c(\phi) \end{bmatrix} \\ &= \begin{bmatrix} c(\psi)c(\theta) & c(\psi)s(\theta)s(\phi) - s(\psi)c(\phi) & c(\psi)s(\theta)c(\phi) + s(\psi)s(\phi) \\ s(\psi)c(\theta) & s(\psi)s(\theta)s(\phi) + c(\psi)c(\phi) & s(\psi)s(\theta)c(\phi) - c(\psi)s(\phi) \\ -s(\theta) & c(\theta)s(\phi) & c(\theta)c(\phi) \end{bmatrix} \end{aligned} \quad (72)$$

ψ represents rotation about the inertial z axis and ϕ is rotation about the body x axis. Body rates are described by Equation 73.

$${}^B \boldsymbol{\omega} = \begin{bmatrix} p & q & r \end{bmatrix}^T \quad (73)$$

Because the rotation matrix is defined for orthonormal bases $R^{-1} = R^T$. Therefore, ${}^B\mathbf{v} = {}^I_B\mathbf{R}(\phi, \theta, \psi)^T {}^I\mathbf{v}$ and ${}^I_B\mathbf{R}(\phi, \theta, \psi)^T = \mathbf{R}_x(\phi)^T \mathbf{R}_y(\theta)^T \mathbf{R}_z(\psi)^T$. Body rates from Equation 73 can also be described by the sum of each Euler rate rotated from its respective frame to the body frame given by Equation 74.

$$\begin{aligned}
{}^B\omega &= {}^B\omega_\phi + {}^B\omega_\theta + {}^B\omega_\psi \\
&= {}^\phi\omega_\phi + \mathbf{R}_x(\phi)^T \omega_\theta + \mathbf{R}_x(\phi)^T \mathbf{R}_y(\theta)^T \omega_\psi \\
&= \begin{bmatrix} \dot{\phi} \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 \\ 0 & c(\phi) & s(\phi) \\ 0 & -s(\phi) & c(\phi) \end{bmatrix} \begin{bmatrix} 0 \\ \dot{\theta} \\ 0 \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 \\ 0 & c(\phi) & s(\phi) \\ 0 & -s(\phi) & c(\phi) \end{bmatrix} \begin{bmatrix} c(\theta) & 0 & -s(\theta) \\ 0 & 1 & 0 \\ s(\theta) & 0 & c(\theta) \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ \dot{\psi} \end{bmatrix} \\
&= \begin{bmatrix} 1 & 0 & -s(\theta) \\ 0 & c(\phi) & s(\phi)c(\theta) \\ 0 & -s(\phi) & c(\phi)c(\theta) \end{bmatrix} \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} \\
&= \mathbf{W}(\phi, \theta) \dot{\eta}
\end{aligned} \tag{74}$$

The ϕ frame is equivalent to the body frame; the θ frame is an intermediate frame just before the last rotation in the ZYX sequence; and the ψ frame occurs before the θ frame in the ZYX sequence as corresponds with Equation 74. To obtain Euler rates the matrix inverse is taken, and the result is given by Equation 75.

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \sin(\phi)\tan(\theta) & \cos(\phi)\tan(\theta) \\ 0 & \cos(\phi) & -\sin(\phi) \\ 0 & \sin(\phi)\sec(\theta) & \cos(\phi)\sec(\theta) \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix} \tag{75}$$

Note that Euler angles face a known issue of singularity when $\theta = 90^\circ$ for an ZYX sequence, causing the determinant of W to become 0.

The standard rigid body equations of motion are given by Equations 76 and 77, where $\omega \times (\mathbf{J}\omega)$ emerges from the Coriolis transport theorem when applying Newton's second law in a rotating frame.

$${}^I\ddot{\mathbf{r}} = \frac{1}{m} \sum {}^I\mathbf{F} \tag{76}$$

$${}^B\dot{\omega} = \mathbf{J}^{-1}(\sum \tau - {}^B\omega \times \mathbf{J}^B\omega) \tag{77}$$

Ignoring the effect of batteries, sensors, etc. on the inertia tensor, the plus configuration causes body axes to be aligned with the principal axes of inertia resulting in Equation 78.

$$\mathbf{J}^{-1} = \begin{bmatrix} 1/I_{xx} & 0 & 0 \\ 0 & 1/I_{yy} & 0 \\ 0 & 0 & 1/I_{zz} \end{bmatrix} \tag{78}$$

The forces in Equation 76 include thrust, gravity, and drag. Drag is approximated by damping proportional to velocity in the body frame with no cross coupling terms. The full translational differential equation is then described by Equation 79 where $\mathbf{C}$ is a diagonal matrix of translational

aerodynamic damping constants.

$${}^I\ddot{\mathbf{r}} = {}^I\mathbf{g} + \frac{1}{m} {}^I_B\mathbf{R}^B\mathbf{T} - \frac{1}{m} {}^I_B\mathbf{R}\mathbf{C}_B^I\mathbf{R}^{TI}\dot{\mathbf{r}} \quad (79)$$

Similarly, Equation 77 can be expanded to Equation 80 where D is a diagonal matrix of rotational aerodynamic damping constants.

$${}^B\dot{\boldsymbol{\omega}} = \mathbf{J}^{-1}({}^B\boldsymbol{\tau}_B - \mathbf{D}^B\boldsymbol{\omega} - {}^B\boldsymbol{\omega} \times \mathbf{J}^B\boldsymbol{\omega}) \quad (80)$$

Thrust and torque can further be described by Equation 81 which is said to hold at steady state [11].

$$\begin{bmatrix} T \\ \tau_{x_B} \\ \tau_{y_B} \\ \tau_{z_B} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & L & 0 & -L \\ -L & 0 & L & 0 \\ c_1 & -c_2 & c_3 & -c_4 \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} \quad (81)$$

B Materials and Methods

The TVLQR and RTI schemes are implemented on the full continuous Quadcopter dynamics with the state vector defined by Equation 82 and control variables by Equation 83.

$$\vec{x} = \begin{bmatrix} r_x \\ r_y \\ r_z \\ \dot{r}_x \\ \dot{r}_y \\ \dot{r}_z \\ \phi \\ \theta \\ \psi \\ p \\ q \\ r \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \\ x_8 \\ x_9 \\ x_{10} \\ x_{11} \\ x_{12} \end{bmatrix} \quad (82)$$

$$\vec{u} = \begin{bmatrix} T & M_x & M_y & M_z \end{bmatrix}^T = \begin{bmatrix} u_1 & u_2 & u_3 & u_4 \end{bmatrix}^T \quad (83)$$

The state temporal derivative, representing all EOMs, is then given by Equation 84.

$$\dot{\vec{x}} = \vec{f}(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \dot{r}_x \\ \dot{r}_y \\ \dot{r}_z \\ \ddot{r}_x \\ \ddot{r}_y \\ \ddot{r}_z \\ \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \\ \dot{p} \\ \dot{q} \\ \dot{r} \end{bmatrix} = \begin{bmatrix} x_4 \\ x_5 \\ x_6 \\ f_4(x_4, x_5, x_6, x_7, x_8, x_9, u_1) \\ f_5(x_4, x_5, x_6, x_7, x_8, x_9, u_1) \\ f_6(x_4, x_5, x_6, x_7, x_8, x_9, u_1) \\ x_{10} + x_{11}\sin(x_7)\tan(x_8) + x_{12}\cos(x_7)\tan(x_8) \\ x_{11}\cos(x_7) - x_{12}\sin(x_7) \\ x_{11}\sin(x_7)\sec(x_8) + x_{12}\cos(x_7)\sec(x_8) \\ \frac{1}{I_{xx}}(u_2 - D_{11}x_{10} + I_{yy}x_{11}x_{12}) \\ \frac{1}{I_{yy}}(u_3 - D_{22}x_{11} - I_{xx}x_{10}x_{12} + I_{zz}x_{10}x_{12}) \\ \frac{1}{I_{zz}}(u_4 - D_{33}x_{12} + I_{xx}x_{10}x_{11}) \end{bmatrix} \quad (84)$$

where:

$$\begin{aligned}
f_4 = & \frac{1}{m} \left(u_1(s(x_7)s(x_9) + c(x_7)c(x_9)s(x_8)) \right. \\
& - x_4 [C_{22}(c(x_7)s(x_9) - c(x_9)s(x_7)s(x_8))(c(x_7)s(x_9) - c(x_9)s(x_7)s(x_8)) \\
& \quad + C_{33}(s(x_7)s(x_9) + c(x_7)c(x_9)s(x_8))(s(x_7)s(x_9) + c(x_7)c(x_9)s(x_8)) \\
& \quad + C_{11}c(x_9)c(x_8)c(x_9)c(x_8)] \\
& + x_5 [C_{22}(c(x_7)c(x_9) + s(x_7)s(x_9)s(x_8))(c(x_7)s(x_9) - c(x_9)s(x_7)s(x_8)) \\
& \quad + C_{33}(c(x_9)s(x_7) - c(x_7)s(x_9)s(x_8))(s(x_7)s(x_9) + c(x_7)c(x_9)s(x_8)) \\
& \quad - C_{11}c(x_8)s(x_9)c(x_9)c(x_8)] \\
& + x_6 [C_{11}s(x_8)c(x_9)c(x_8) - C_{33}c(x_7)c(x_8)(s(x_7)s(x_9) + c(x_7)c(x_9)s(x_8)) \\
& \quad + C_{22}c(x_8)s(x_7)(c(x_7)s(x_9) - c(x_9)s(x_7)s(x_8))] \Big)
\end{aligned}$$

$$\begin{aligned}
f_5 = & \frac{1}{m} \left(-u_1(c(x_9)s(x_7) - c(x_7)s(x_9)s(x_8)) \right. \\
& + x_4 [C_{22}(c(x_7)s(x_9) - c(x_9)s(x_7)s(x_8))(c(x_7)c(x_9) + s(x_7)s(x_9)s(x_8)) \\
& \quad + C_{33}(s(x_7)s(x_9) + c(x_7)c(x_9)s(x_8))(c(x_9)s(x_7) - c(x_7)s(x_9)s(x_8)) \\
& \quad - C_{11}c(x_9)c(x_8)c(x_8)s(x_9)] \\
& - x_5 [C_{22}(c(x_7)c(x_9) + s(x_7)s(x_9)s(x_8))(c(x_7)c(x_9) + s(x_7)s(x_9)s(x_8)) \\
& \quad + C_{33}(c(x_9)s(x_7) - c(x_7)s(x_9)s(x_8))(c(x_9)s(x_7) - c(x_7)s(x_9)s(x_8)) \\
& \quad + C_{11}c(x_8)s(x_9)c(x_8)s(x_9)] \\
& + x_6 [C_{33}c(x_7)c(x_8)(c(x_9)s(x_7) - c(x_7)s(x_9)s(x_8)) \\
& \quad - C_{22}c(x_8)s(x_7)(c(x_7)c(x_9) + s(x_7)s(x_9)s(x_8)) \\
& \quad + C_{11}s(x_8)c(x_8)s(x_9)] \Big)
\end{aligned}$$

$$\begin{aligned}
f_6 = & \frac{1}{m} \left(u_1c(x_7)c(x_8) - gm \right. \\
& - x_4 [-C_{11}c(x_9)c(x_8)s(x_8) - C_{22}c(x_7)s(x_9)c(x_8)s(x_7) + C_{33}s(x_7)s(x_9)c(x_7)c(x_8) \\
& \quad + C_{33}c(x_7)c(x_9)s(x_8)c(x_7)c(x_8) + C_{22}c(x_9)s(x_7)s(x_8)c(x_8)s(x_7)] \\
& - x_5 [-C_{11}c(x_8)s(x_9)s(x_8) + C_{22}c(x_7)c(x_9)c(x_8)s(x_7) - C_{33}c(x_9)s(x_7)c(x_7)c(x_8) \\
& \quad + C_{33}c(x_7)s(x_9)s(x_8)c(x_7)c(x_8) + C_{22}s(x_7)s(x_9)s(x_8)c(x_8)s(x_7)] \\
& - x_6 [C_{11}s(x_8)s(x_8) + C_{33}c(x_7)c(x_8)c(x_7)c(x_8) + C_{22}c(x_8)s(x_7)c(x_8)s(x_7)] \Big)
\end{aligned}$$

The full state space EOMs, constraints, and everything else was defined in *QuadcopterOfflineCasADI* in *OCP496* [8] for generating an optimal reference trajectory offline, utilizing the open source direct collocation and interior point trajectory optimizer from CasADI <https://web.casadi.org/>. After the reference trajectory was computed, it was used to initialize controllers for simulation. All of the computation and simulation carried out herein can be reproduced by running *QuadcopterOfflineCasADI*

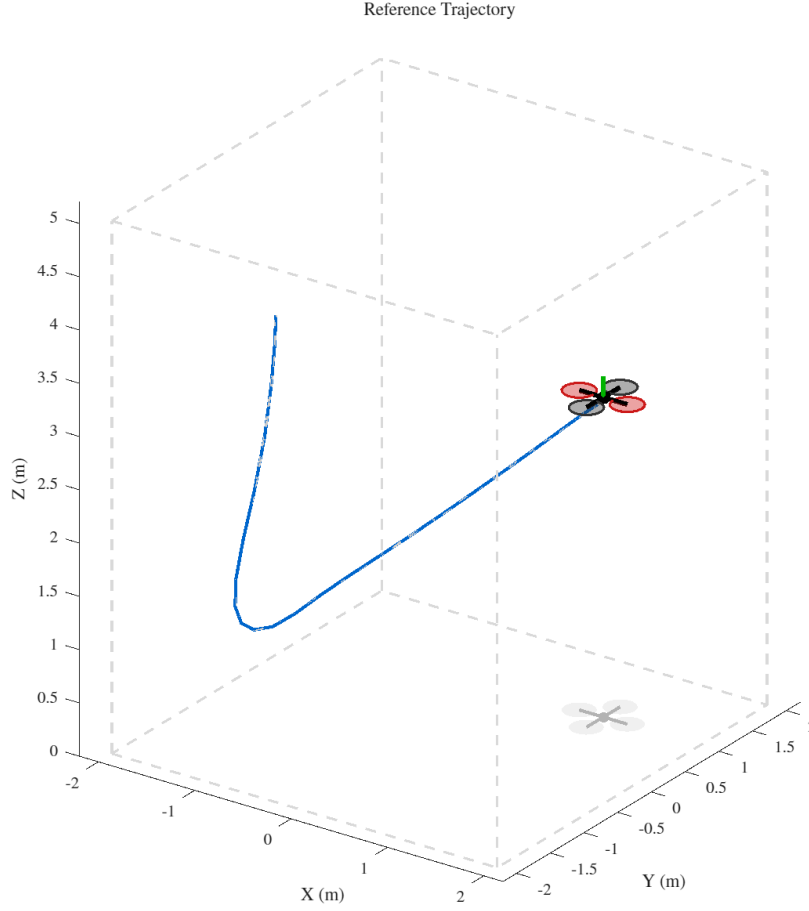


Figure 2: Trajectory animation screenshot

with functions from *OCP496* [8] downloaded in the same directory.

Video animations of the simulated quadcopter trajectories, comparing the stabilization behaviors of the respective controllers under severe disturbances, are openly available for review at [8]. A screenshot of a given animation is given by Figure 2.

B.1 Experimental Design

To rigorously evaluate the robustness and viability of the aforementioned control strategies, a 250 run Monte Carlo simulation was conducted. The environment was designed to subject the quadcopter to both parametric uncertainty and continuous environmental disturbances.

Parametric uncertainty was injected by applying a $\pm 5 - 10\%$ standard deviation to the nominal values of the quadcopter’s mass, inertia tensor (I_{xx}, I_{yy}, I_{zz}), and aerodynamic drag coefficients prior to each flight. Due to the omission of lower level controllers and dynamics realistic of true hardware, actuator noise was applied as an additive zero-mean Gaussian disturbance, $\sigma = 5\% \cdot u_{\max}$, to the unconstrained virtual control inputs prior to integration. Furthermore, to simulate realistic, chaotic wind, a continuous sum-of-sines process disturbance was applied directly to the translational acceleration states within the *ode45* integration loop. The base wind direction was

uniformly randomized between 0 and 2π radians for each run to prevent directional bias, ensuring the ensemble average of the disturbance vectors converged to zero. Additionally, repeatability was ensured by introducing a random number generator seed.

The controllers evaluated included TVLQR, Shrinking Horizon Linear MPC, Shrinking Horizon RTI NMPC, and Receding Horizon RTI NMPC. Actuator limits were strictly defined at $T_{max} = 2.4$ N and $T_{min} = 0$ N per rotor. Upon reaching the end of the predefined trajectory horizon all controllers, aside from Receding RTI, transitioned to a standard LQR hover-hold policy (Mode 2) to evaluate steady-state regulation. In fact, both Shrinking Horizon controllers were transitioned to Mode 2 four time steps prior to the completion of the original horizon due to instability and infeasibility caused by the vanishingly small horizon.

B.2 Statistical Analysis

Due to the severity of the combined disturbances, a subset of simulation trajectories became divergent, resulting in virtual boundary contact or unbounded state explosion. To prevent asymptotic divergence from artificially skewing the visual trajectory averages, the Thrust Violations, Quadratic Cost, and Pareto analyses are displayed exclusively over the success population, but were rigorously accounted for in the State Violations and Distance to Target figures.

C Results

C.1 Trade-off Analysis and Pareto Optimality

The Pareto front (Figure 3) illustrates the fundamental trade-off between total absolute control effort ($\Sigma \mathbf{u}^T \mathbf{u}$) and terminal tracking accuracy under severe disturbances. The TVLQR controller exhibits strong disturbance rejection and terminal accuracy, comparatively clustered near the origin of the Y-axis, though variation is large. However, it incurs a severe, often erratic actuation cost spread across the X-axis. Conversely, the Linear MPC demonstrates an inability to overcome sustained nonlinear disturbances, yielding severe terminal position errors and Control violations when switched to Mode 2 from a state of large error. The proposed RTI NMPC formulations (Shrinking and Receding horizons) occupy the optimal region of the trade-off curve, successfully bounding terminal error while maintaining highly efficient and consistent control effort, though outliers exist for both—likely due to Gaussian perturbations not being truncated.

C.2 Total Trajectory Cost Evaluation

To strictly quantify the aggregate performance of each control policy, the total quadratic cost—defined by the discrete sum of the state and input deviations from the terminal hover state stage costs over the trajectory horizon was evaluated. As depicted in Figure 4, all controllers showed similar optimality. Though TVLQR tended to have the lowest objective cost, it resulted in the most control command violations as expected—due to small control weighting matrix values the TVLQR cost was able to stay small. Shrinking Horizon Linear MPC showed similar Quadratic Cost to both RTI NMPC controllers but had a larger number of outliers.

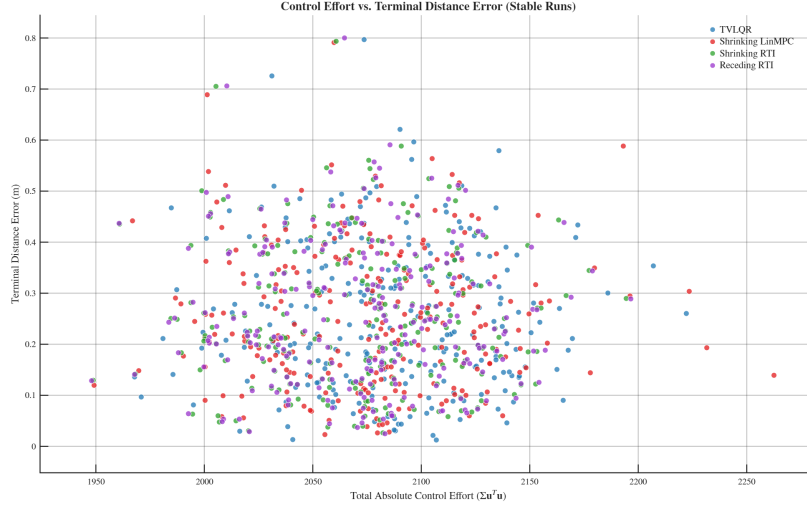


Figure 3: Pareto front illustrating Control Effort vs. Terminal Distance Error for the successful simulation population. RTI NMPC controllers comparatively cluster optimally near the bottom-left.

C.3 Trajectory Tracking and Disturbance Rejection

Figure 5 details the mean Euclidean distance to the target state over time. Upon transitioning to the LQR hover-hold at the end of the trajectory, the controllers settle at a minor steady-state offset (≈ 0.5 m). This offset is expected, as the randomized continuous wind pushes the vehicle off-center, and standard LQR lacks the integral action required to achieve zero steady-state error under constant disturbance, though this work could easily be extended to include integral action. The Receding RTI variation was found to continue increasing because it does not transition to the LQR controller and may convergence to an alternate terminal state in the presence of perturbations. As expected, Shrinking RTI deviation tracks Receding RTI until it activates Mode 2. Linear Shrinking Horizon MPC also performs as expected, quickly amassing a large deviation before switching to Mode 2.

C.4 Constraint Satisfaction and Safety Rate

While TVLQR achieved the highest trajectory completion rate (97.6%) 6, analysis of the control inputs reveals a critical flaw in its physical viability. As shown in Figure 7, TVLQR frequently commanded individual rotor thrusts drastically exceeding the physical limits of the motors (exceeding T_{max} by significant margins) to force the system to equilibrium. Because standard LQR lacks inequality constraint awareness, it relies on "infinite" control authority to survive gusts. Conversely, the Receding RTI architecture successfully navigated 91.2% of the extreme disturbance scenarios while strictly adhering to the 2.4 N actuator limits, recording exactly zero constraint violations.

D Discussion

The results of this study highlight a vital paradigm in optimal control: theoretical stabilization does not equate to physical viability. The unconstrained nature of TVLQR allows it to exhibit math-

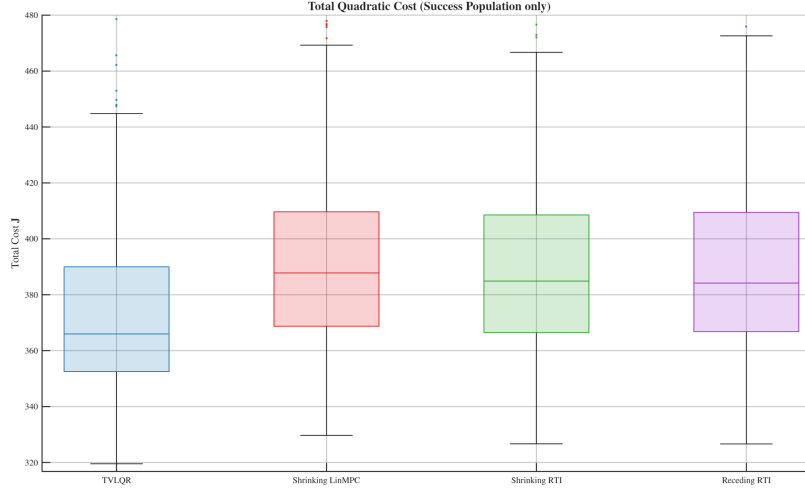


Figure 4: Comparison of the total accumulated quadratic cost for each controller across the successful Monte Carlo runs.

ematically superior variance and success rates; however, it achieves this by demanding impossible actuator dynamics. Deploying such a controller on physical quadcopter hardware would result in immediate motor saturation, loss of control authority, and subsequent catastrophic failure.

In contrast, the Real-Time Iteration (RTI) NMPC architectures demonstrated robust tracking and optimal energy expenditure while rigorously respecting the physical boundaries of the system. While the nonlinear predictive controllers yielded a slightly lower theoretical success rate in simulation (91.2%), their strict adherence to actuator constraints proves them vastly superior for real-world implementation.

A Numerical Integration and Finite Differencing

Numerical integration can convert continuous dynamics, Equation 2, to discrete dynamics, 5. Many numerical integration schemes exist, each offering differing computational complexity and accuracy. This paper considers only direct integrators due to their simplicity and popularity—implicit integrators also exist which can improve solution accuracy per compute time, but they are omitted for because a multitude of existing research exists which can be readily substituted for the method mentioned herein. The simplest numerical integrator is known as Forward Euler, which is defined in Equation 85.

$$f_k(x_k, u_k) = x_{k+1} \approx x_k + dt f(x_k, u_k) \quad (85)$$

Forward Euler achieves speed at the expense of accuracy; it assumes the state derivate is constant during the dt time interval. To improve accuracy Second Order Runge Kutta (RK2) can be employed as defined in Equation 86.

$$\begin{aligned} k_1 &= f(x_k, u_k) \\ k_2 &= f\left(x_k + k_1 \frac{dt}{2}, u_k\right) \\ f_k(x_k, u_k) &= x_{k+1} \approx x_k + k_2 dt \end{aligned} \quad (86)$$

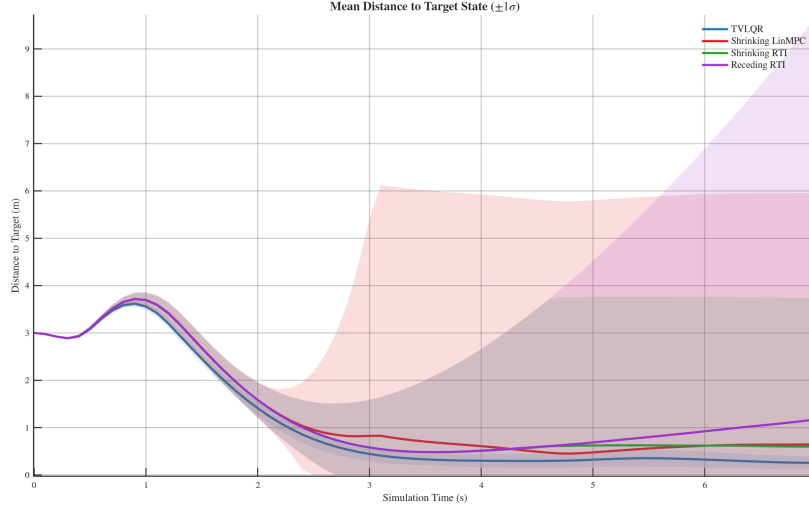


Figure 5: Mean distance to the target state ($\pm 1\sigma$) over time. The transition to a steady-state offset at the end of the horizon reflects the LQR hover-hold balancing against continuous wind.

RK2 sacrifices speed for accuracy. If more accuracy is desired Fourth Order Runge Kutta (RK4) is a commonly employed direct numerical integrator due to its higher accuracy compared to Forward Euler and RK2; the formula is given by Equation 87.

$$\begin{aligned}
 k_1 &= f(x_k, u_k) \\
 k_2 &= f\left(x_k + k_1 \frac{dt}{2}, u_k\right) \\
 k_3 &= f\left(x_k + k_2 \frac{dt}{2}, u_k\right) \\
 k_4 &= f(x_k + k_3 dt, u_k) \\
 f_k(x_k, u_k) &= x_{k+1} \approx x_k + (k_1 + 2k_2 + 2k_3 + k_4) \frac{dt}{6}
 \end{aligned} \tag{87}$$

RK4 is of course the most computationally expensive of the aforementioned direct methods, but is also potentially the most popular.

Acknowledgments

Many thanks to Dr. Wu, professor in the Department of Mechanical and Aerospace Engineering at North Carolina State University, for advising this work. His support in allowing me to explore my own interests was invaluable.

Funding

This research did not receive funding, and is the result of two successive semesters on research for academic credit within the Department of Mechanical and Aerospace Engineering at North Carolina State University.

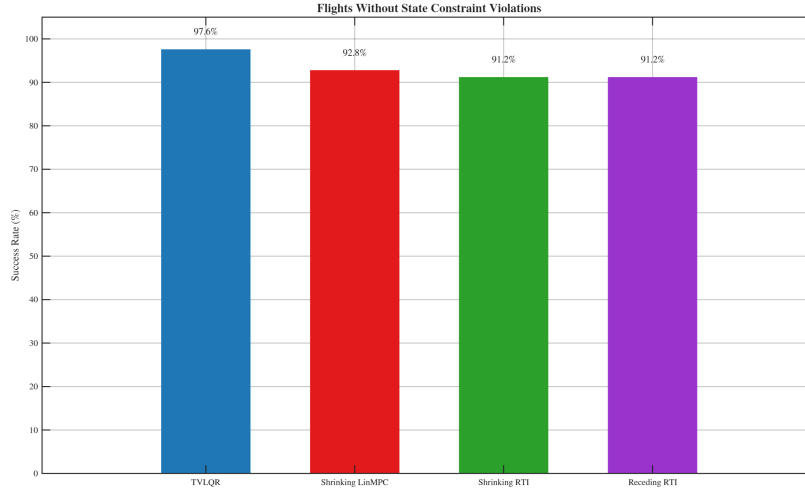


Figure 6: Magnitude of individual rotor thrust violations. TVLQR massively exceeds physical motor limits to achieve stabilization, whereas MPC strictly bounds inputs.

Conflicts of Interest

The author declares that there is no conflict of interest regarding the publication of this article.

Data Availability

All code and data to replicate the results herein is located in the author’s GitHub repository *OCP496* [8].

References

1. Bryson A. Optimal Control-1950 to 1985. *IEEE Control Systems Magazine* 1996;16:26–33.
2. Bennett S. A Brief History of Automatic Control. *IEEE Control Systems Magazine* 2018;1996:17–25.
3. Brunton SL, Proctor JL, and Kutz JN. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences* 2016;113:3932–7.
4. Yu R and Wang R. Learning dynamical systems from data: An introduction to physics-guided deep learning. *Proceedings of the National Academy of Sciences* 2024;121:e2311808121.
5. Wang C and Song Z. Convex Model Predictive Control for Rocket Vertical Landing. In: *2018 37th Chinese Control Conference (CCC)*. 2018:9837–42. DOI: 10.23919/ChiCC.2018.8483147.
6. Eaton JW and Rawlings JB. Model-Predictive Control of Chemical Processes. *Chemical Engineering Science* 1992;47:705–20.
7. Gros S, Zanon M, Quirynen R, Bemporad A, and Diehl M. From linear to nonlinear MPC: bridging the gap via the real-time iteration. *International Journal of Control* 2016;93:1–19.
8. Kiziah. MAE496-Research-2026. <https://github.com/lukekiziah/OCP496>. 2026.

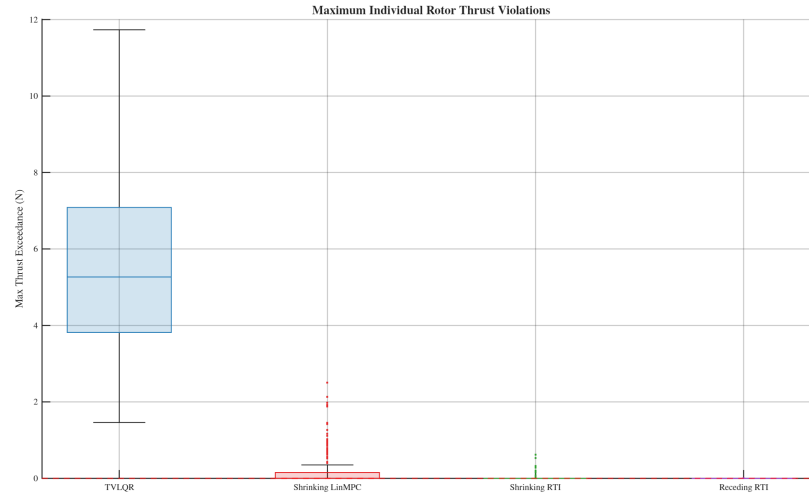


Figure 7: Magnitude of individual rotor thrust violations. TVLQR massively exceeds physical motor limits to achieve stabilization, whereas MPC strictly bounds inputs—though switching to Mode 2 caused violations.

9. Stengel R. Optimal Control and Estimation. Dover books on advanced mathematics. Dover Publications, 1994.
10. Nguyen T, Xuan Mung N, and Hong SK. Quadcopter Adaptive Trajectory Tracking Control: A New Approach via Backstepping Technique. Applied Sciences 2019;9:3873.
11. Martins L, Cardeira C, and Oliveira P. Linear Quadratic Regulator for Trajectory Tracking of a Quadrotor. IFAC-PapersOnLine 2019;52:176–81.